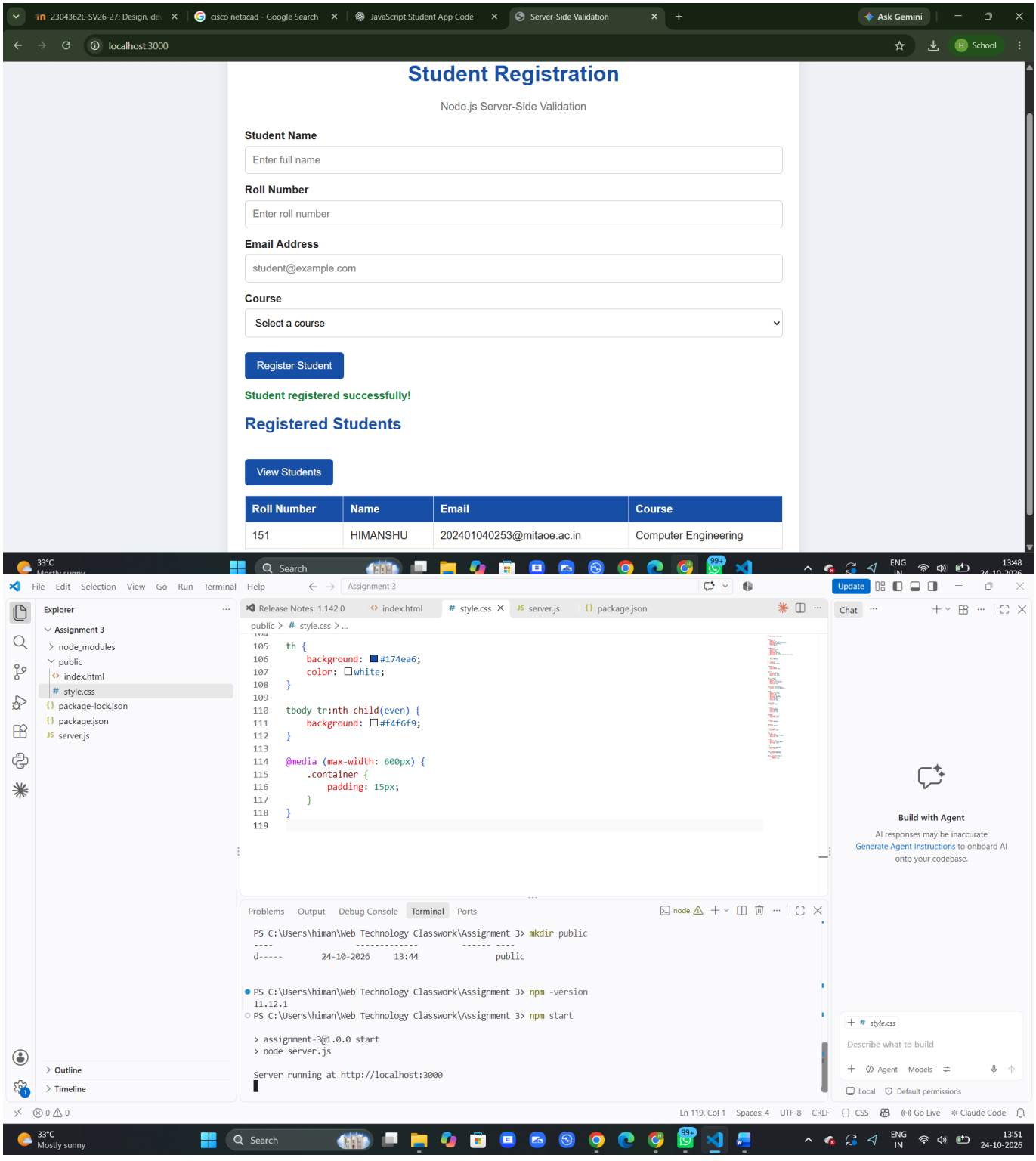
Node.js Server-Side Validation

Complete Source Code — Student Registration Web Application

Name - Himanshu Sahajwani
Prn – 202401040253

This application demonstrates server-side validation using Node.js and Express. The server validates submitted student data before saving it to an in-memory array.


Student Registration
Node.js Server-Side Validation
Student Name
Enter full name
Roll Number
Enter roll number
Email Address
student@example.com
Course
Select a course
Register Student
Student registered successfully!
Registered Students
View Students
Roll Number
Name
Email
Course
151
HIMANSHU
202401040253@mitaoe.ac.in
Computer Engineering
th {
background: #174ea6;
color: white;
}
tbody tr:nth-child(even) {
background: #f4f6f9;
}
@media (max-width: 600px) {
.container {
padding: 15px;
}
}
PS C:\Users\himan\Web Technology Classwork\Assignment 3> mkdir public
d----- 24-10-2026 13:44 public
PS C:\Users\himan\Web Technology Classwork\Assignment 3> npm -version
11.12.1
PS C:\Users\himan\Web Technology Classwork\Assignment 3> npm start
> assignment-3@1.0.0 start
> node server.js
Server running at http://localhost:3000

server.js

File name: server.js

```
const express = require("express");
const path = require("path");

const app = express();
const PORT = process.env.PORT || 3000;

// Middleware
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Serve HTML and CSS files
app.use(express.static(path.join(__dirname, "public")));

// Allowed course names
const allowedCourses = [
    "Computer Engineering",
    "Information Technology",
    "Mechanical Engineering",
    "Electronics Engineering"
];

// Array to store registered students (temporary storage)
const students = [];

// Server-side validation function
function validateStudent(data) {
    const errors = {};

    const name = typeof data.name === "string"
        ? data.name.trim().replace(/\s+/g, " ")
        : "";

    const roll = typeof data.roll === "string"
        ? data.roll.trim()
        : "";

    const email = typeof data.email === "string"
        ? data.email.trim()
        : "";

    const course = typeof data.course === "string"
        ? data.course.trim()
        : "";

    // Validate name
    if (name.length < 3 || name.length > 50) {
        errors.name = "Name must be between 3 and 50 characters.";
    } else if (!/^[\p{L}][\p{L}\s.'-]*$/u.test(name)) {
        errors.name = "Name must contain valid letters.";
    }

    // Validate roll number
    if (!/^[A-Za-z0-9-]{1,20}$/.test(roll)) {
        errors.roll = "Roll number must contain letters, numbers or hyphens.";
    } else if (students.some(s =>
        s.roll.toLowerCase() === roll.toLowerCase()
```

```
    )) {
        errors.roll = "This roll number is already registered.";
    }

    // Validate email
    if (
        email.length > 254 ||
        !/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)
    ) {
        errors.email = "Please enter a valid email address.";
    } else if (students.some(s =>
        s.email.toLowerCase() === email.toLowerCase()
    )) {
        errors.email = "This email is already registered.";
    }

    // Validate course
    if (!allowedCourses.includes(course)) {
        errors.course = "Please select a valid course.";
    }

    return {
        errors,
        student: { name, roll, email, course }
    };
}

// POST route to register a student
app.post("/register", (req, res) => {
    const { errors, student } = validateStudent(req.body || {});

    // Reject invalid data on the server
    if (Object.keys(errors).length > 0) {
        return res.status(400).json({
            success: false,
            message: "Validation failed. Please correct the errors.",
            errors
        });
    }

    // Save only after validation succeeds
    students.push(student);

    return res.status(201).json({
        success: true,
        message: "Student registered successfully!",
        student
    });
});

// Route to view registered students
app.get("/students", (req, res) => {
    res.json({
        success: true,
        count: students.length,
        students
    });
});

// Handle unknown routes
app.use((req, res) => {
    res.status(404).json({
```

```
        success: false,
        message: "Route not found."
    });
});

// Start server
app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);
});
```

package.json

File name: package.json

```
{
  "name": "server-side-validation",
  "version": "1.0.0",
  "description": "Student registration demonstrating server-side validation using Node.js",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "node server.js"
  },
  "dependencies": {
    "express": "^5.1.0"
  }
}
```

public/index.html

File name: public/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Server-Side Validation</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <div class="container">
        <h1>Student Registration</h1>
        <p class="subtitle">Node.js Server-Side Validation</p>

        <form id="studentForm">
            <label for="name">Student Name</label>
            <input type="text" id="name" name="name"
                   placeholder="Enter full name" required>
            <small id="nameError" class="error"></small>

            <label for="roll">Roll Number</label>
            <input type="text" id="roll" name="roll"
                   placeholder="Enter roll number" required>
            <small id="rollError" class="error"></small>

            <label for="email">Email Address</label>
            <input type="email" id="email" name="email"
                   placeholder="student@example.com" required>
            <small id="emailError" class="error"></small>

            <label for="course">Course</label>
            <select id="course" name="course" required>
                <option value="">Select a course</option>
                <option value="Computer Engineering">Computer Engineering</option>
                <option value="Information Technology">Information Technology</option>
                <option value="Mechanical Engineering">Mechanical Engineering</option>
                <option value="Electronics Engineering">Electronics Engineering</option>
            </select>
            <small id="courseError" class="error"></small>

            <button type="submit">Register Student</button>
        </form>

        <p id="message" role="status" aria-live="polite"></p>

        <h2>Registered Students</h2>
        <button type="button" id="viewStudents">View Students</button>

        <div class="table-wrapper">
            <table>
                <thead>
                    <tr>
                        <th>Roll Number</th>
                        <th>Name</th>
                        <th>Email</th>
                        <th>Course</th>
                    </tr>
                </thead>
```

```
            <tbody id="studentTable">
                <tr>
                    <td colspan="4">Click View Students to load records.</td>
                </tr>
            </tbody>
        </table>
    </div>
</div>

<script>
    const form = document.getElementById("studentForm");
    const message = document.getElementById("message");

    const fields = ["name", "roll", "email", "course"];

    function clearErrors() {
        fields.forEach(field => {
            document.getElementById(field + "Error").textContent = "";
        });

        message.textContent = "";
        message.className = "";
    }

    function showErrors(errors) {
        fields.forEach(field => {
            document.getElementById(field + "Error").textContent =
                errors[field] || "";
        });
    }

    form.addEventListener("submit", async function (event) {
        event.preventDefault();
        clearErrors();

        const data = {
            name: document.getElementById("name").value,
            roll: document.getElementById("roll").value,
            email: document.getElementById("email").value,
            course: document.getElementById("course").value
        };

        try {
            const response = await fetch("/register", {
                method: "POST",
                headers: {
                    "Content-Type": "application/json"
                },
                body: JSON.stringify(data)
            });

            const result = await response.json();

            if (!response.ok) {
                showErrors(result.errors || {});
                message.textContent =
                    result.message || "Registration failed.";
                message.className = "failure";
                return;
            }
```

```
                message.textContent = result.message;
                message.className = "success";

                form.reset();
                await loadStudents();

            } catch (error) {
                message.textContent =
                    "Unable to connect to the server. Please try again.";
                message.className = "failure";
            }
        });

        async function loadStudents() {
            const table = document.getElementById("studentTable");

            try {
                const response = await fetch("/students");
                const result = await response.json();

                if (!response.ok) {
                    throw new Error("Could not load students.");
                }

                table.replaceChildren();

                if (result.students.length === 0) {
                    const row = table.insertRow();
                    const cell = row.insertCell();
                    cell.colSpan = 4;
                    cell.textContent = "No students registered yet.";
                    return;
                }

                result.students.forEach(student => {
                    const row = table.insertRow();

                    [student.roll, student.name,
                     student.email, student.course].forEach(value => {
                        const cell = row.insertCell();
                        cell.textContent = value;
                    });
                });

            } catch (error) {
                message.textContent = "Unable to load student records.";
                message.className = "failure";
            }
        }

        document.getElementById("viewStudents")
            .addEventListener("click", loadStudents);
    </script>

</body>
</html>
```

public/style.css

File name: public/style.css

```
* {
    box-sizing: border-box;
}

body {
    margin: 0;
    padding: 25px 15px;
    font-family: Arial, sans-serif;
    background: #eef2f7;
    color: #222;
}

.container {
    max-width: 850px;
    margin: auto;
    padding: 25px;
    background: white;
    border-radius: 10px;
    box-shadow: 0 4px 15px rgba(0, 0, 0, 0.1);
}

h1, h2 {
    color: #174ea6;
}

h1, .subtitle {
    text-align: center;
}

.subtitle {
    color: #666;
    margin-bottom: 25px;
}

label {
    display: block;
    font-weight: bold;
    margin: 15px 0 6px;
}

input, select {
    width: 100%;
    padding: 11px;
    border: 1px solid #ccc;
    border-radius: 5px;
    font-size: 15px;
}

input:focus, select:focus {
    outline: 2px solid #8bb7ff;
}

button {
    margin-top: 18px;
    padding: 11px 18px;
    border: none;
    border-radius: 5px;
    background: #174ea6;
```

```
    color: white;
    cursor: pointer;
    font-size: 15px;
}

button:hover {
    opacity: 0.85;
}

.error {
    display: block;
    color: #d93025;
    margin-top: 5px;
    font-size: 13px;
}

#message {
    font-weight: bold;
    margin-top: 15px;
}

.success {
    color: #188038;
}

.failure {
    color: #d93025;
}

.table-wrapper {
    overflow-x: auto;
}

table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 15px;
}

th, td {
    padding: 10px;
    border: 1px solid #ddd;
    text-align: left;
}

th {
    background: #174ea6;
    color: white;
}

tbody tr:nth-child(even) {
    background: #f4f6f9;
}

@media (max-width: 600px) {
    .container {
        padding: 15px;
    }
}
```